ITBA 2026

Clase 3: Arquitectura

Fer Cargnelutti

What NOT to do...

Functional

Discipline imposed upon assignment (mutability)

Object Oriented

Discipline imposed upon indirect transfer of control

Structured

Discipline imposed upon direct transfer of control

1957 — 1966 — 1968

Clean Architecture

The architect responsibilities

El paradigma de programación

El lenguaje de programación

Los frameworks

Los patrones de diseño

La base de datos

La interfaz del sistema

El lenguaje de documentación

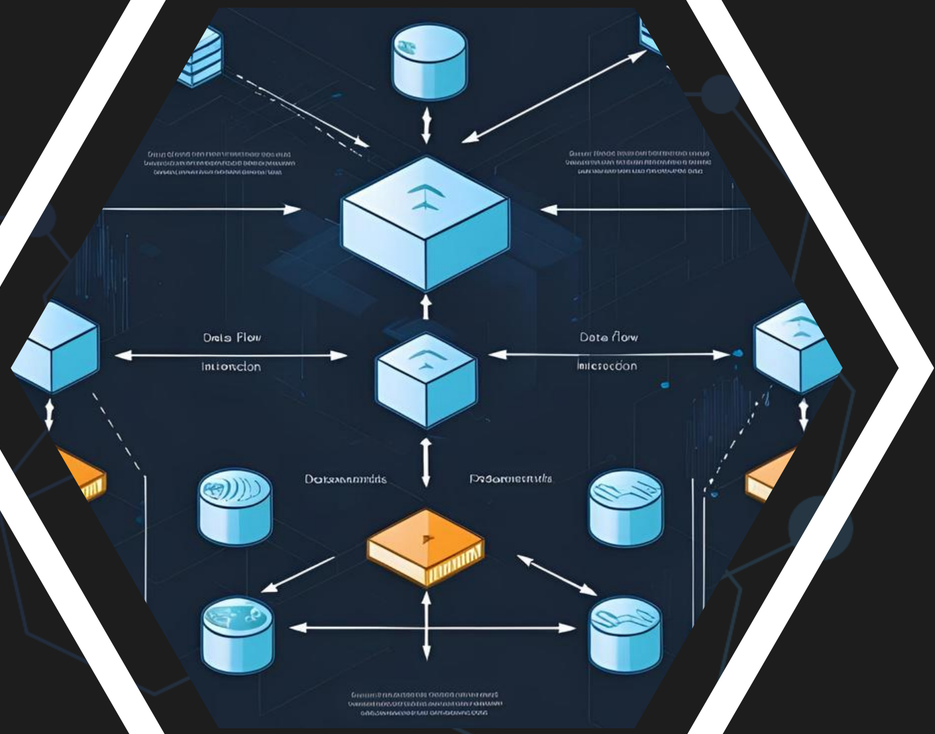
La metodología de Desarrollo

El HW para despliegue

The architect responsibilities

El rol del arquitecto es posponer las decisiones el mayor tiempo posible

Software values



Structure

Functionality

Software is meant to be soft

Mantener abiertas

la mayor cantidad

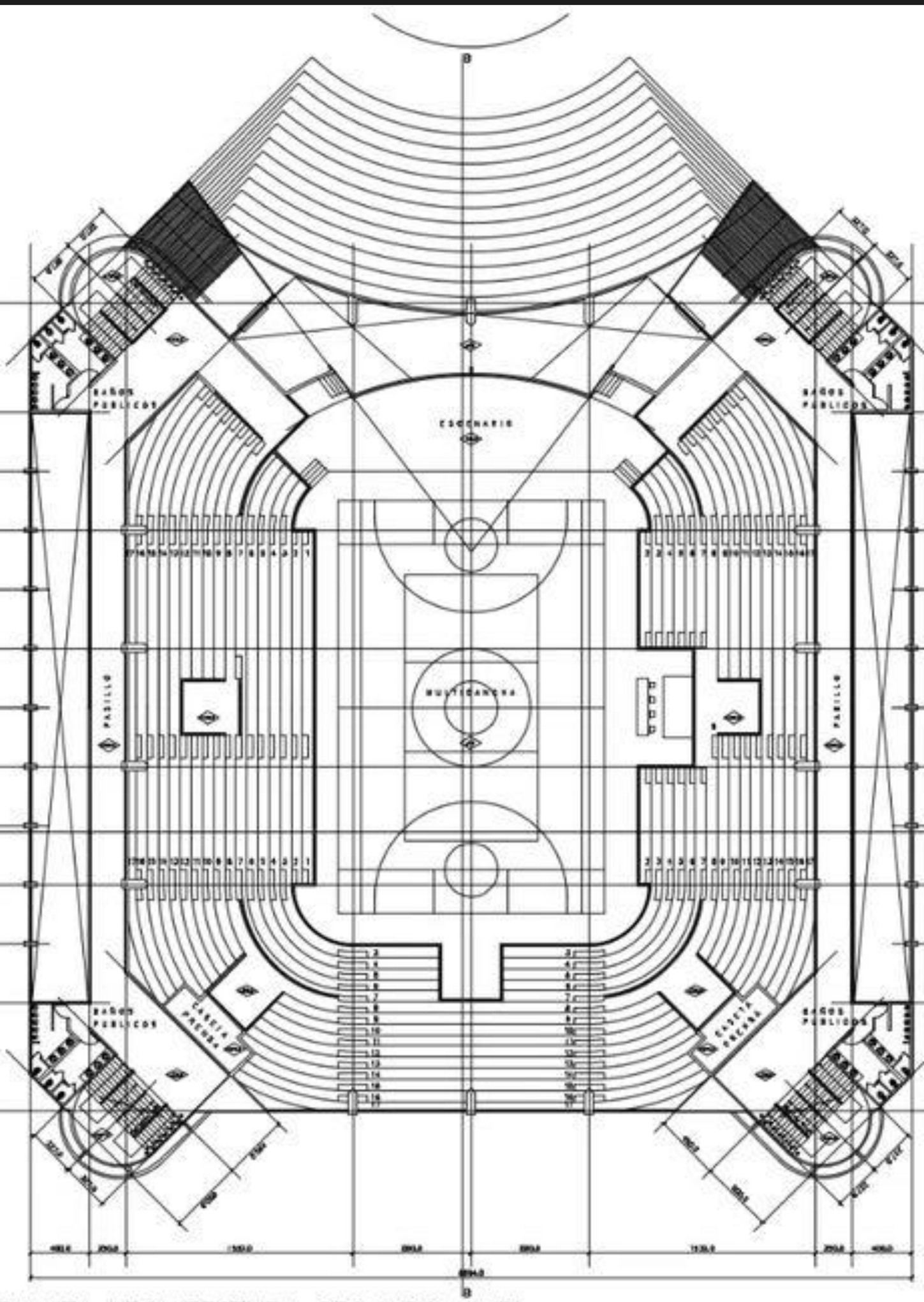
de opciones durante

el mayor tiempo posible

Software decomposition

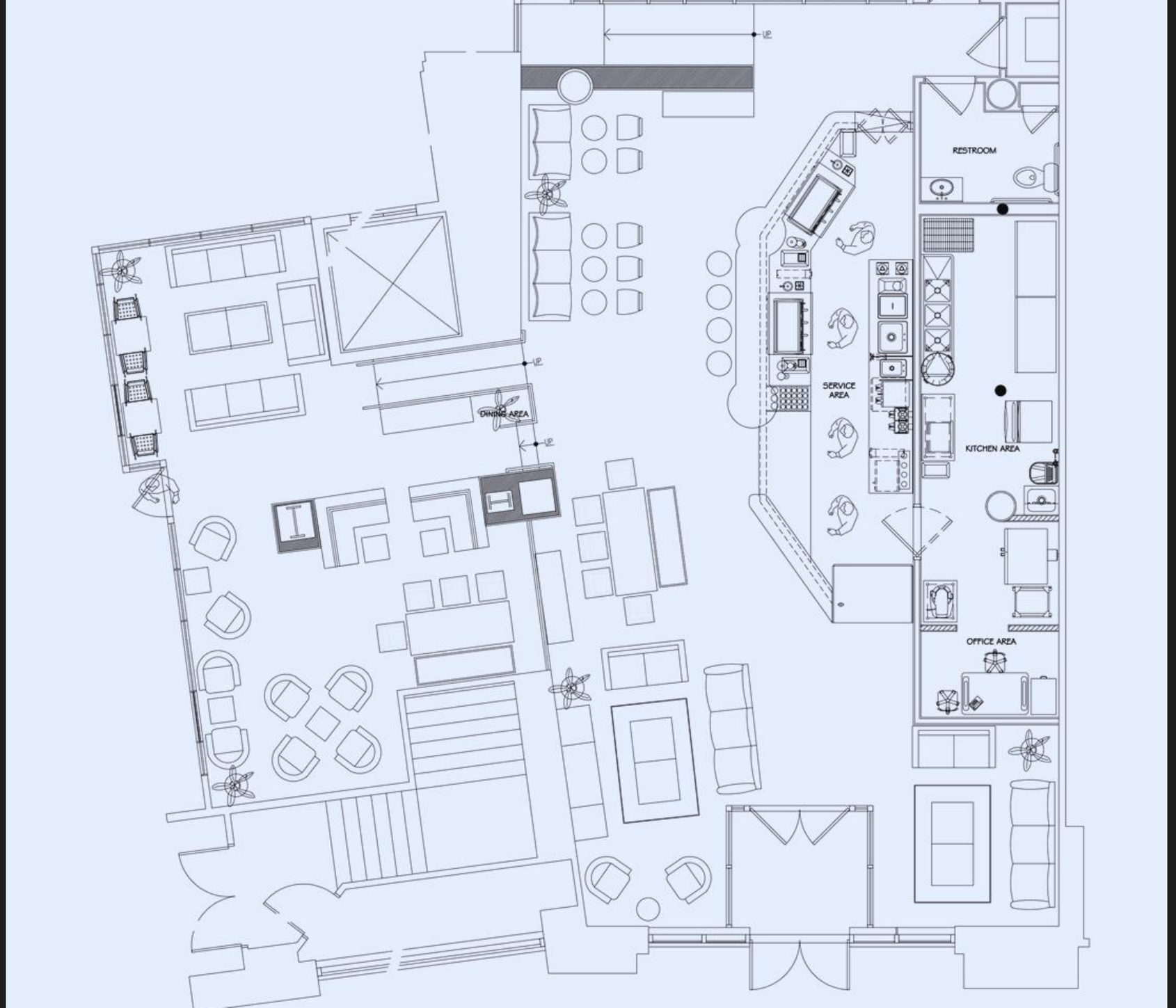
Policies

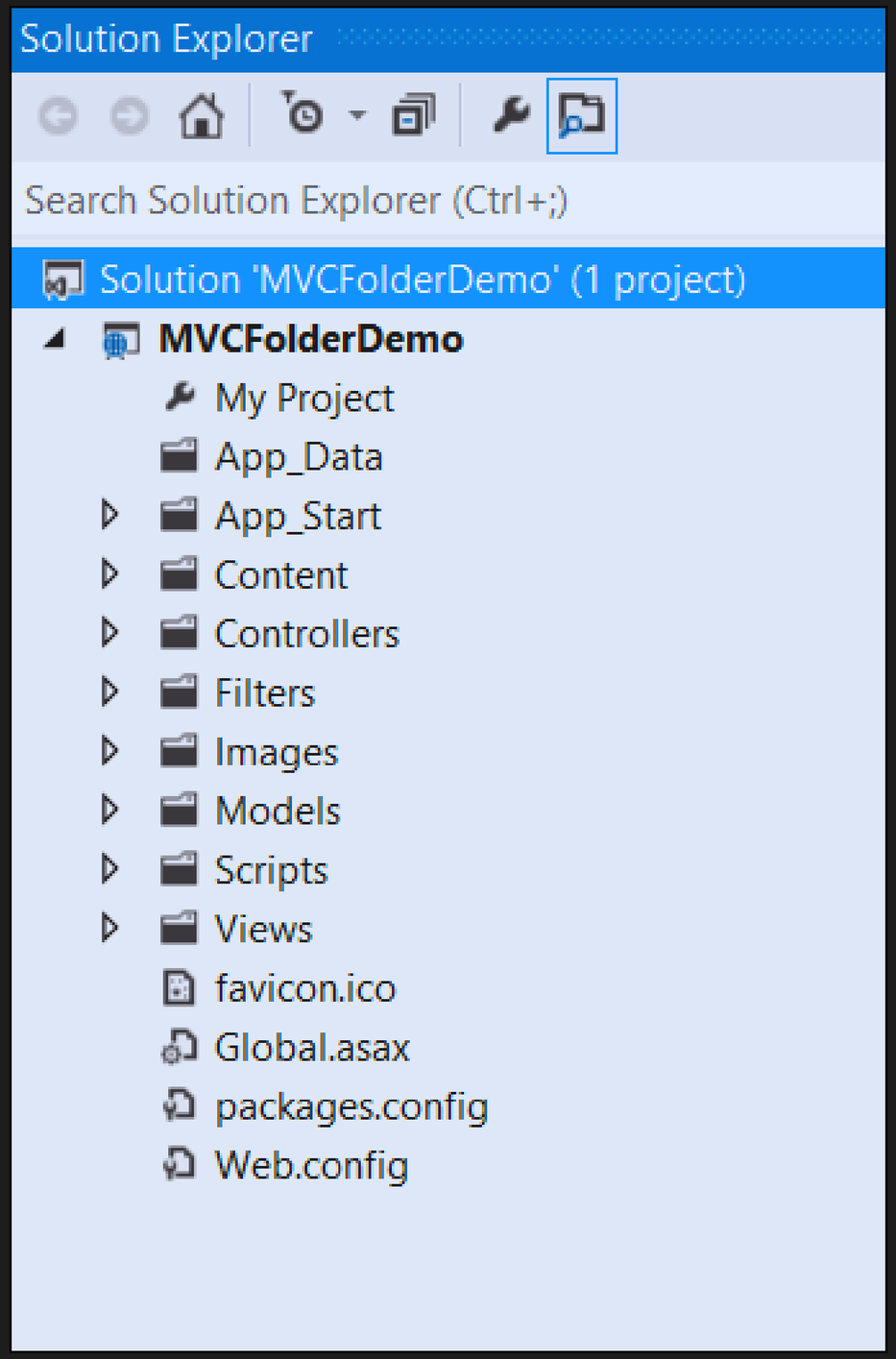
Details

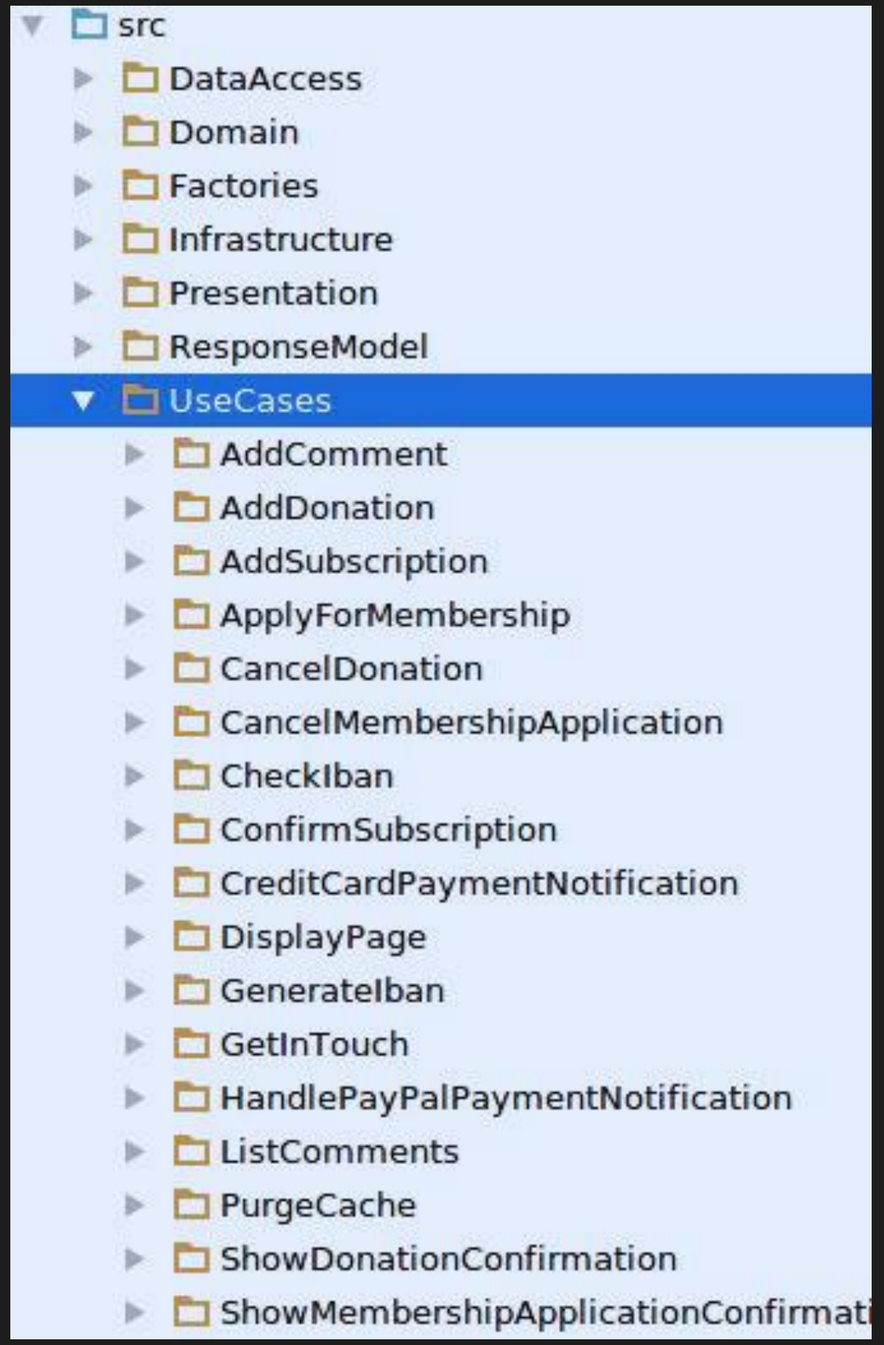


PLANTA ARQUITECTURA SEGUNDO NIVEL

ESCALA: 1/200


UP
RESTROOM
SERVICE AREA
KITCHEN AREA
DINING AREA
UP
UP
OFFICE AREA


Solution Explorer
Search Solution Explorer (Ctrl+;)
Solution 'MVCFolderDemo' (1 project)
MVCFolderDemo
My Project
App_Data
App_Start
Content
Controllers
Filters
Images
Models
Scripts
Views
favicon.ico
Global.asax
packages.config
Web.config

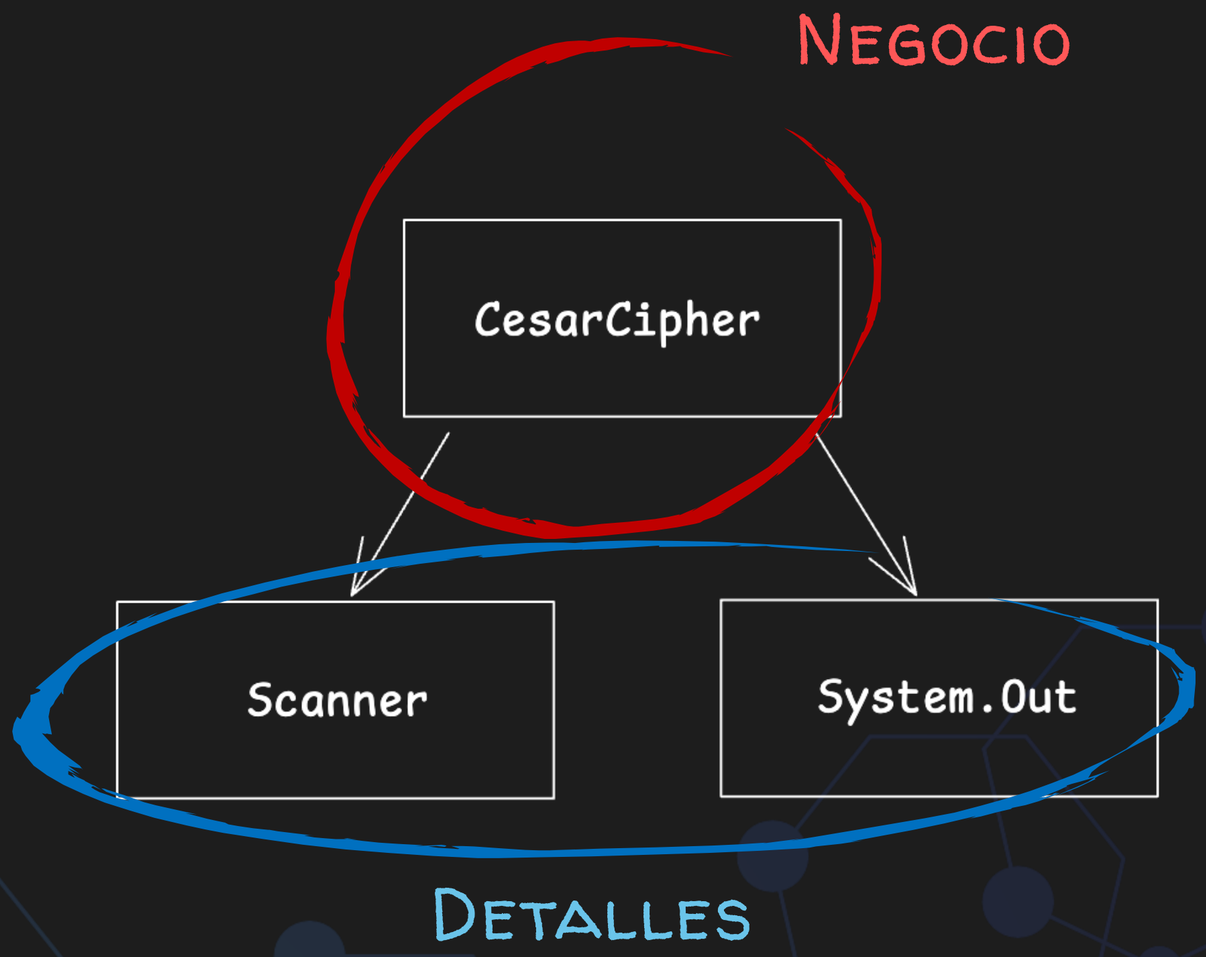

src
DataAccess
Domain
Factories
Infrastructure
Presentation
ResponseModel
UseCases
AddComment
AddDonation
AddSubscription
ApplyForMembership
CancelDonation
CancelMembershipApplication
CheckIban
ConfirmSubscription
CreditCardPaymentNotification
DisplayPage
GenerateIban
GetInTouch
HandlePayPalPaymentNotification
ListComments
PurgeCache
ShowDonationConfirmation
ShowMembershipApplicationConfirmati

And again...

Dependencies

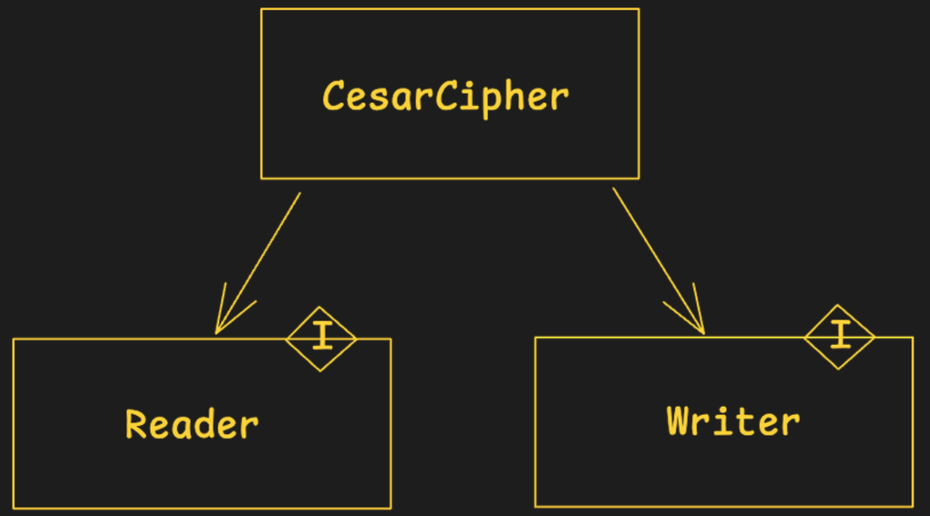
Cipher example

```java
public class CesarCipher {
    public void encrypt() {
        Scanner input = new Scanner(System.in);
        String textToEncrypt = input.nextLine();

        String encryptedText = cesarCipher(textToEncrypt);

        System.out.println("Original text: " + textToEncrypt);
        System.out.println("Encrypted text: " + encryptedText);
    }

    private String cesarCipher(String text) {...}
}
```



Depend on interfaces

```java
public class CesarCipher {
    private final Reader reader;
    private final Writer writer;

    public CesarCipher(Reader reader, Writer writer) {
        this.reader = reader;
        this.writer = writer;
    }

    public void encrypt() {
        String textToEncrypt = reader.readLine();
        String encryptedText = cesarCipher(textToEncrypt);
        writer.writeLine("Original text: " + textToEncrypt);
        writer.writeLine("Encrypted text: " + encryptedText);
    }

    private String cesarCipher(String text) {...}
}
```

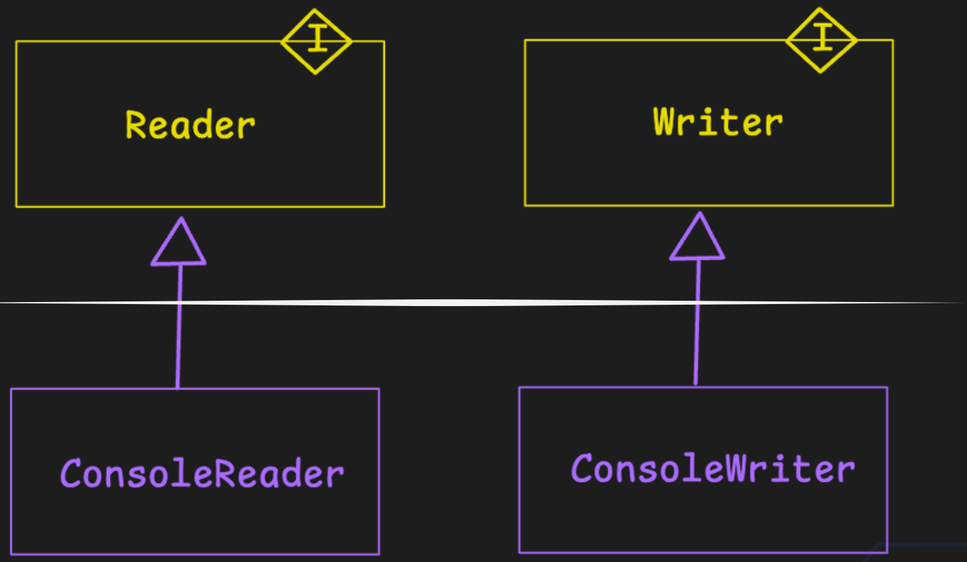


Los detalles dependen de las reglas de negocio

Details and policies

```java
package edu.cleanarch.cesarcipher.io;

import edu.cleanarch.cesarcipher.entities.Reader;
import java.util.Scanner;

public class ConsoleReader implements Reader {
    @Override
    public String readLine() {
        return new Scanner(System.in).nextLine();
    }
}
```

```java
package edu.cleanarch.cesarcipher.io;

import edu.cleanarch.cesarcipher.entities.Writer;

public class ConsoleWriter implements Writer {
    @Override
    public void writeLine(String line) {
        System.out.println(line);
    }
}
```

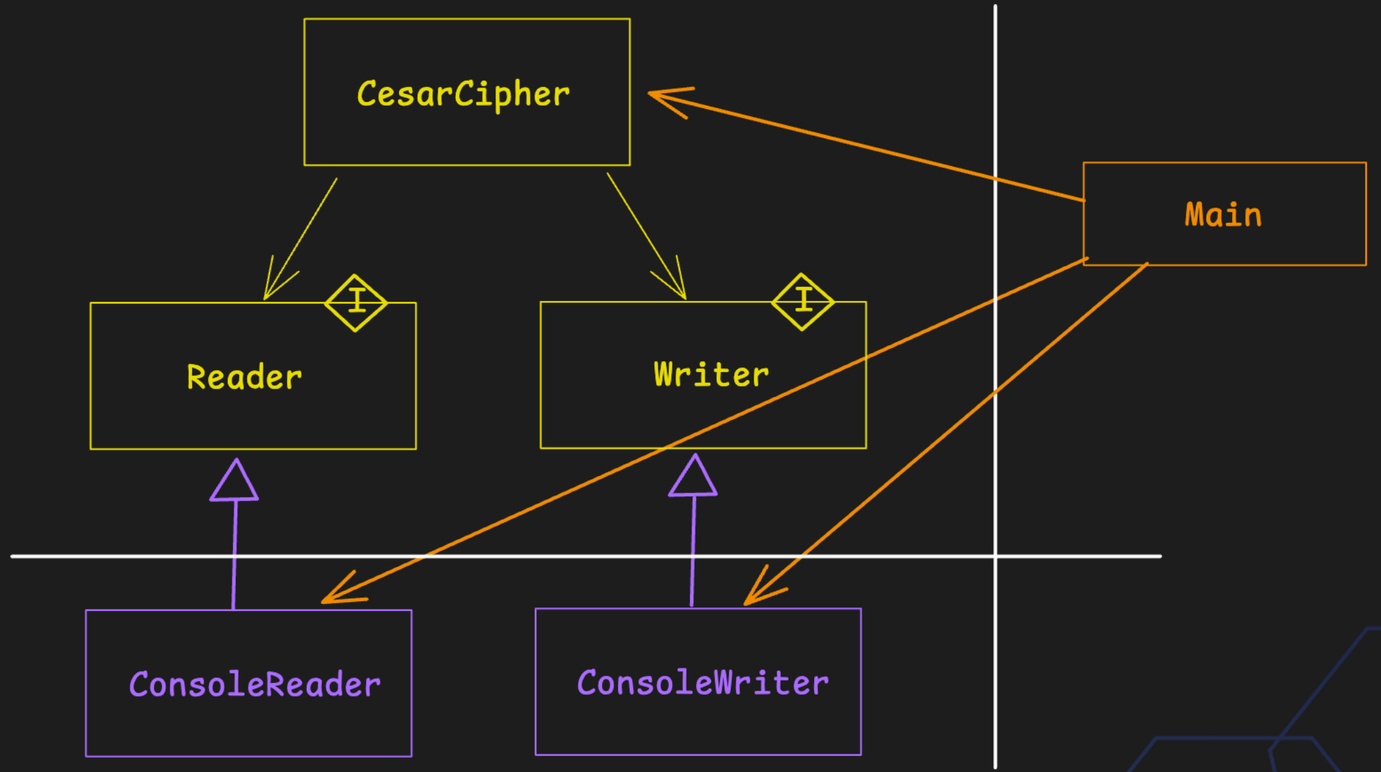


El componente Main

```java
package edu.cleanarch.cesarcipher.main;

import edu.cleanarch.cesarcipher.entities.CesarCipher;
import edu.cleanarch.cesarcipher.entities.Reader;
import edu.cleanarch.cesarcipher.entities.Writer;
import edu.cleanarch.cesarcipher.io.ConsoleReader;
import edu.cleanarch.cesarcipher.io.ConsoleWriter;

public class Main {
    public static void main(String[] args) {
        Writer writer = new ConsoleWriter();
        Reader reader = new ConsoleReader();

        CesarCipher cipher = new CesarCipher(reader, writer);
        cipher.encrypt();
    }
}
```

El componente Main

```java
package edu.cleanarch.cesarcipher.main;

import edu.cleanarch.cesarcipher.entities.CesarCipher;
import edu.cleanarch.cesarcipher.entities.Reader;
import edu.cleanarch.cesarcipher.entities.Writer;
import edu.cleanarch.cesarcipher.io.ConsoleReader;
import edu.cleanarch.cesarcipher.io.ConsoleWriter;

public class Main {
    public static void main(String[] args) {
        Writer writer = new ConsoleWriter();
        Reader reader = new ConsoleReader();

        CesarCipher cipher = new CesarCipher(reader, writer);
        cipher.encrypt();
    }
}
```



Details and business